



Disciplina: Lab. WEB I

Aluno: Robson Lisboa Santos

Docente: Eduardo Teles

PatrimWeb

Camaçari

2025.2

SUMÁRIO

1. APRESENTAÇÃO	3
2. TECNOLOGIAS UTILIZADAS	3
3. PLATAFORMA DE IMPLEMENTAÇÃO	4
4. SISTEMA.....	5
4.1 CAMADAS E PADRÃO DO PROJETO	7
4.1.1 Modelo	10
4.1.2 Visão	11
4.1.3 Controlador	12
4.2 TELAS	14

1. APRESENTAÇÃO

O sistema PatrimWeb é uma aplicação web destinada ao gerenciamento de patrimônio organizacional, com foco no controle de equipamentos, unidades, fabricantes, usuários e movimentações entre setores de uma instituição. Sua finalidade é centralizar o registro e a rastreabilidade de ativos, oferecendo histórico detalhado e relatórios gerenciais que apoiam auditorias e a tomada de decisão.

A temática é relevante porque organizações frequentemente enfrentam baixa visibilidade sobre localização de equipamentos, dificuldade de rastrear movimentações e ausência de histórico de manutenção ou deslocamentos. O sistema busca mitigar esses problemas por meio de uma plataforma integrada, com autenticação, controle de sessão e funcionalidades específicas para inventário, movimentações e relatórios.

2. TECNOLOGIAS UTILIZADAS

Decidiu-se implementar o sistema PatrimWeb utilizando a linguagem de programação Java, o banco de dados relacional MySQL a IDE Eclipse para uso do Java Servlets e Inteligência Artificial no Auxiliar de Criação das Views em HTML5, CSS e JavaScript.

- Linguagens de marcação
 - HTML5 (páginas JSP estruturadas com marcação HTML, como index.jsp, dashboard.jsp, equipamentos.jsp, etc.).
 - Linguagens de estilo
 - CSS3 (arquivo patrimweb.css responsável pela estilização global das interfaces).
 - Linguagens de programação
 - Java (back-end, servlets, modelos, DAOs e utilitários, compatível com Tomcat 9 e Java 8+).
 - JavaScript (vanilla, para interações na interface, validações e comportamentos de navegação).
 - Frameworks e bibliotecas
 - Java Servlet API (javax.servlet) para implementação de servlets, controle de sessão e processamento HTTP.
 - JSP (JavaServer Pages) como tecnologia de visão para renderização dinâmica de páginas.
 - JSTL (JavaServer Pages Standard Tag Library) para lógica de apresentação e iteração em JSP.
 - MySQL Connector/J 9.2.0 como driver JDBC para comunicação com o banco MySQL.
 - Banco de dados utilizado
 - MySQL 8.0 (banco relacional, com suporte a transações ACID e chaves estrangeiras).
-
- ✓ Linguagens de estilo e marcação:
 - HTML: Linguagem de marcação (estrutura do conteúdo); e,
 - CSS: Linguagem de estilo (aparência do conteúdo).
 - ✓ Linguagens de programação: Java

- ✓ Frameworks ou plataformas: Eclipse EE.
- ✓ Gerência de Dados: MySQL 8.0.

3. PLATAFORMA DE IMPLEMENTAÇÃO

O sistema foi desenvolvido como aplicação web Java, utilizando um ambiente de desenvolvimento integrado e um servidor de aplicação compatível com Servlets e JSP.

- IDE utilizada
 - Eclipse IDE, conforme evidenciado pelos metadados de projeto e estrutura padrão Java Web (src/main/java, src/main/webapp, diretório Servers).
- Servidor
 - Apache Tomcat 9.0.110, configurado no diretório Servers e compatível com Servlet API 4.0 e JSP 2.3.
- Banco de dados
 - MySQL 8.0, configurado por meio do arquivo database.properties localizado em lib/configuracao, com parâmetros de conexão JDBC.
- Sistema operacional ou ambiente de execução
 - Ambiente Java padrão (JVM) em servidor de aplicação Tomcat, em contexto típico de desenvolvimento local (localhost:8080/patrimweb) e passível de implantação em servidores Windows ou Linux.
- Outras ferramentas relevantes
 - Git (versionamento) com repositório remoto no GitHub.
 - Inclusão manual de bibliotecas em WEB-INF/lib para gerenciamento de dependências, incluindo o driver MySQL.

4. SISTEMA

O banco de dados do PatrimWeb foi modelado em estilo relacional, contemplando as principais entidades: usuario, fabricante, unidade, equipamento e movimentacao. As tabelas são conectadas por chaves primárias numéricas e chaves estrangeiras que garantem integridade referencial entre fabricantes, unidades, equipamentos e registros de movimento.

As entidades principais são:

- Tabela USUARIO
 - Armazena os dados dos usuários do sistema, incluindo identificação, dados pessoais, credenciais de acesso e metadados de inserção.
 - Campos: id_usu (PK), nome_usu, cpf_usu, telefone_usu, email_usu, endereco_usu, data_insercao, login_google, senha_usu, id_perfil.
 - Função: representar agentes que acessam o sistema, podendo ser operadores, administradores ou auditores.
- Tabela FABRICANTE
 - Armazena os fabricantes dos equipamentos cadastrados.
 - Campos: id_fab (PK), nome_fab, data_insercao.
 - Função: permitir que equipamentos sejam associados a um fabricante específico, viabilizando relatórios por origem ou marca.
- Tabela UNIDADE
 - Armazena as unidades organizacionais (setores, departamentos, locais físicos).
 - Campos: id_unid (PK), nome_unid, telefone_unid, email_unid, endereco_unid, data_insercao.
 - Função: representar locais onde equipamentos podem estar alocados ou entre os quais podem ser movimentados.
- Tabela EQUIPAMENTO
 - Armazena os equipamentos sob controle patrimonial.
 - Campos: id_equip (PK), nome_equip, num_serie_equip, fabricante (FK), data_insercao.
 - Função: caracterizar cada bem patrimonial, vinculando-o a um fabricante.
- Tabela MOVIMENTACAO
 - Registra as movimentações de equipamentos entre unidades.
 - Campos: id_mov (PK), equipamento (FK), fabricante (FK), tipo_movimentacao, unidade_origem (FK), usuario_origem (FK), unidade_destino (FK), usuario_destino (FK), observação, data_insercao.
 - Função: manter o histórico de deslocamentos de cada equipamento, permitindo rastreabilidade temporal e espacial.
- Tabela PERFIL
 - Armazena os perfis que serão atribuídos aos usuários cadastrados.

- Campos: id (PK), nome.
- Função: permitir que perfis sejam associados a usuários do sistema.
- Tabela PERMISSAO
 - Armazena as permissões que serão atribuídos aos perfis cadastrado.
 - Campos: id (PK), modulo, acao, descrição.
 - Função: permitir que permissões sejam associados a perfis cadastrado.

Relações e cardinalidades:

- FABRICANTE (1) — (N) EQUIPAMENTO: um fabricante pode ter muitos equipamentos, e cada equipamento referência exatamente um fabricante por meio de id_fabricante.
- PERFIL (N) — (N) PERMISSAO: um perfil pode ter muitas permissões, e uma permissão referência vários perfis por meio da entidade associativa perfil_permissao.
- PERFIL (1) — (N) PERMISSAO: um perfil pode ter muitos usuários, e cada usuário referência exatamente um perfil por meio de id_perfil.
- UNIDADE DE ORIGEM (1) — (N) MOVIMENTACAO: uma unidade de origem pode estar em muitas movimentações, e cada movimentação está associado a uma unidade de origem via unidade_origem.
- USUARIO DE ORIGEM (1) — (N) MOVIMENTACAO: um usuário de origem pode estar em muitas movimentações, e cada movimentação está associado a um usuário de origem via usuario_origem.
- UNIDADE DE DESTINO (1) — (N) MOVIMENTACAO: uma unidade de destino pode estar em muitas movimentações, e cada movimentação está associado a uma unidade de destino via unidade_destino.
- USUARIO DE DESTINO (1) — (N) MOVIMENTACAO: um usuário de destino pode estar em muitas movimentações, e cada movimentação está associado a um usuário de destino via usuario_destino.
- EQUIPAMENTO (1) — (N) MOVIMENTACAO: um equipamento pode estar em muitas movimentações, e cada movimentação está associada a um equipamento via equipamento.
- FABRICANTE (1) — (N) MOVIMENTACAO: um fabricante pode estar em muitas movimentações, e cada movimentação está associado a um fabricante via fabricante.

Integração dos dados no sistema:
Os dados são manipulados pela camada de persistência (DAOs), que executa operações CRUD sobre essas tabelas e devolve objetos de modelo às camadas de controle e visão. A integridade do modelo é reforçada por restrições de chaves estrangeiras, como a impossibilidade de excluir fabricantes com equipamentos associados, o que mantém a consistência do histórico de patrimônio.

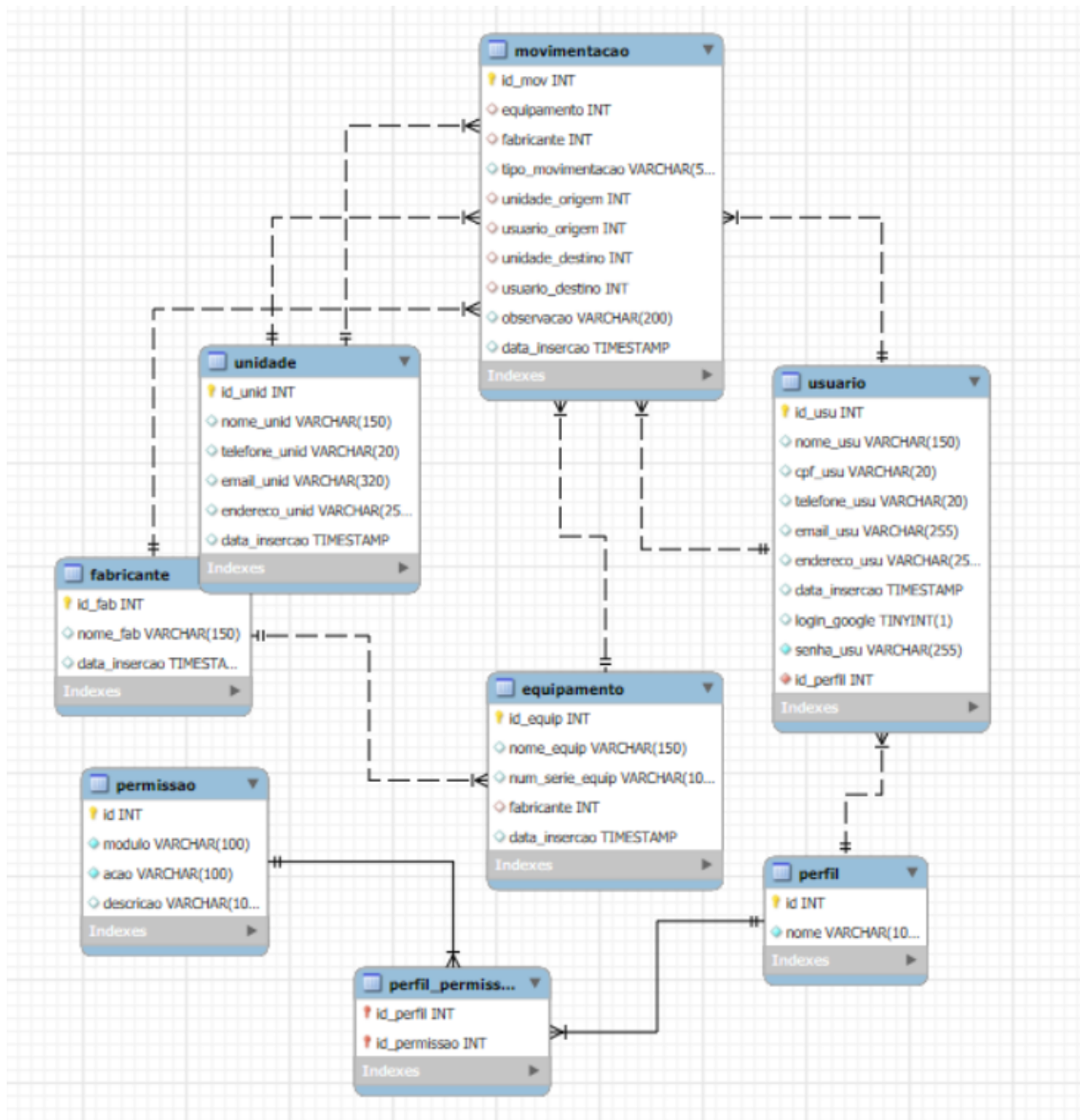


Figura 1 – DER do Banco de Dados MySQL

4.1 CAMADAS E PADRÃO DO PROJETO

A aplicação adota o padrão arquitetural MVC (Model–View–Controller) organizado em três camadas lógicas: apresentação, controle/negócio e persistência. Essa arquitetura estabelece separação clara de responsabilidades, favorecendo manutenção, evolução e testes.

- Camada de Apresentação (View)
 - Implementada principalmente com JSP, HTML, CSS e JavaScript, localizada em src/main/webapp (ex.: index.jsp, pages/dashboard.jsp, pages/equipamentos.jsp, pages/fabricantes.jsp, pages/usuarios.jsp, pages/unidades, pages/movientacoes.jsp, pages/perfil.jsp, pages/permissão.jsp, pages/relatorio_equipamentos.jsp, pages/relatorio_fabricantes.jsp, pages/ relatorio_usuarios.jsp, pages/relatorio_unidades, pages/ relatorio_movientacoes.jsp).
 - Responsável por exibir dados ao usuário, receber entradas via formulários e renderizar interfaces com apoio de JSTL.
- Camada de Controle (Controller)

- Constituída por servlets Java, alocados em `br.com.patrimweb.controller` (IndexController, LoginController, DashboardController, EquipamentoController, FabricanteController, UsuarioController, UnidadeController, MovimentacaoController, PerfilController, PermissaoController, LoginController, LogoutController, LoginGoogleController, ConfiguracaoController, RelatorioEquipamentoController, RelatorioUnidadeController, RelatorioUsuarioController, RelatorioMovimentacaoController, Seguranca).
- Responsável por receber requisições HTTP, validar sessão, interpretar ações (por parâmetro action), coordenar chamadas a DAOs e encaminhar respostas para as views.
- Camada de Modelo/Persistência (Model)
 - Composta por classes de modelo em `br.com.patrimweb.model` (Usuario, Equipamento, Fabricante, Unidade, Movimentacao, Perfil, Permissao) e classes de acesso a dados em `br.com.patrimweb.dao` (DAOGenerico, UsuarioDAO, EquipamentoDAO, FabricanteDAO, UnidadeDAO, MovimentacaoDAO, PerfilDAO, PermissaoDAO, PerfilPermissaoDAO).
 - Responsável por representar as entidades de domínio e encapsular a lógica de acesso ao banco de dados via JDBC.

Árvore de projeto:

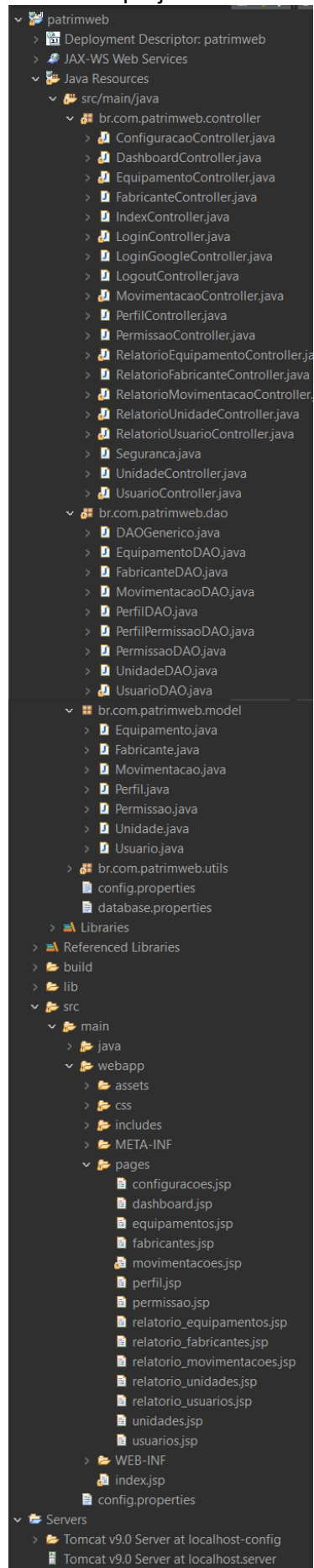


Figura 2 – Projeto implementado na IDE Eclipse

A seguir, apresentamos um descritivo da composição das camadas.

4.1.1 Modelo

As classes Model representam entidades do domínio e são utilizadas para transportar dados entre a camada de persistência e a camada de controle. Em geral, cada tabela do banco possui uma classe POJO (Plain Old Java Object - Ou seja, é **uma classe Java simples**, criada apenas para **representar dados**, sem depender de frameworks ou regras complexas) correspondente, com atributos privados, construtores e métodos de acesso (getters/setters).

- Classe Usuario.java
 - Objetivo: representar um usuário do sistema, incluindo identificação, dados pessoais e credenciais.
 - Dados representados: idUsu, nomeUsu, cpfUsu, telefoneUsu, emailUsu, enderecoUsu, dataInsercao, loginGoogle, senhaUsu, id_perfil.
 - Comunicação: utilizada por UsuarioDAO para leitura/escrita no banco e pelos controllers (especialmente LoginController e UsuarioController) para validações, autenticação e exibição na interface.
- Classe Fabricante.java
 - Objetivo: representar um fabricante de equipamentos.
 - Dados representados: idFab, nomeFab, data_insercao.
 - Comunicação: manipulada por FabricanteDAO e FabricanteController, além das views que associam equipamentos a fabricantes.
- Classe Unidade.java
 - Objetivo: representar uma unidade organizacional, como setor ou departamento.
 - Dados representados: id_unid, nome_unid, telefone_unid, email_unid, endereço_unid, data_insercao.
 - Comunicação: utilizada por UnidadeDAO, UnidadeController e telas relacionadas a alocação e movimentação de equipamentos.
- Classe Equipamento.java
 - Objetivo: representar um equipamento sob controle patrimonial.
 - Dados representados: id_e Equip, nome Equip, num_serie Equip, associações a Fabricante além de data_inserção.
 - Comunicação: utilizada por EquipamentoDAO e EquipamentoController, além de ser referenciada por Movimentacao e relatórios, permitindo carregar fabricante e unidade relacionados.

- Classe Movimentacao.java
 - Objetivo: representar um registro de movimentação de equipamento entre unidades.
 - Dados representados: id_mov, referência ao equipamento, referência a fabricante, tipo_movimentacao, unidade de origem, usuarios_origem, unidade de destino, usuario_destino, observacao e data_insercao.
 - Comunicação: utilizada por MovimentacaoDAO e MovimentacaoController, além das views de histórico e relatórios de movimentações.
- Classe Perfil.java
 - Objetivo: representar um registro de perfil de usuário.
 - Dados representados: id, nome.
 - Comunicação: utilizada por PerfilDAO, além da view Perfil.
- Classe Pmissao.java
 - Objetivo: representar um registro de permissao de usuário.
 - Dados representados: id, modulo, acao, descricao.
 - Comunicação: utilizada por PmissaoDAO, além das views Permissao e Configurações.

Além disso, a classe Conexao.java (em utils) abstrai a criação de conexões JDBC, com leitura de database.properties, sendo utilizada pelos DAOs para obter objetos Connection

4.1.2 Visão

A camada View é composta por páginas JSP e recursos web que implementam a interface com o usuário. Os arquivos situados em src/main/webapp inclui index.jsp, e em src/main/webapp/pages incluem dashboard.jsp, equipamentos.jsp, fabricantes.jsp, usuarios.jsp, unidades.jsp, movimentacoes.jsp, configuracoes.jsp, perfis.jsp, permissões.jsp, relatorio_equipamentos.jsp, relatorio_fabricantes.jsp, relatorio_usuarios.jsp, relatorio_unidades.jsp, relatorio_movimentacoes.jsp, além de arquivos de apoio como patrimweb.css.

Principais telas e funções:

- index.jsp (Tela de Login)
 - Função: realizar a autenticação inicial do usuário, capturando credenciais e encaminhando-as ao LoginController.
 - Conexão com controlador: formulário HTML com método POST direcionado para /LoginController, com campos de usuário/email e senha.
- dashboard.jsp (Dashboard)
 - Função: apresentar visão geral de estatísticas (totais de equipamentos, movimentações e usuários) e fornecer navegação central para os demais módulos.

- Conexão com controlador: recebe atributos de contexto preenchidos por DashboardController e disponibiliza links para outros controllers (EquipamentoController, UsuarioController, etc).
- equipamentos.jsp (Gerenciamento de Equipamentos)
 - Função: listar, filtrar e realizar operações CRUD sobre equipamentos, com formulários modais para inclusão e edição.
 - Conexão com controlador: utiliza dados fornecidos por EquipamentoController (lista de equipamentos, fabricantes, unidades) e envia requisições GET/POST com parâmetro action para adicionar, editar ou excluir registros.
- fabricantes.jsp (Fabricantes)
 - Função: administrar fabricantes, permitindo cadastro, edição e exclusão, com validação de vínculos com equipamentos.
 - Conexão com controlador: interage com FabricanteController por meio de ações definidas em formulários e links na tabela.
- usuarios.jsp (Usuários)
 - Função: gerenciar usuários do sistema, com filtros por período, geração de relatórios e operações CRUD.
 - Conexão com controlador: integrando-se ao UsuarioController e RelatorioController para carregar usuários e exportar dados.
- unidades.jsp (Unidades)
 - Função: cadastrar e manter unidades organizacionais que servirão de origem/destino para equipamentos e movimentações.
 - Conexão com controlador: utiliza UnidadeController para listar, criar, editar e excluir unidades.
- movimentacoes.jsp (Movimentações)
 - Função: registrar e visualizar o histórico de movimentações de equipamentos entre unidades.
 - Conexão com controlador: interage com MovimentacaoController, que fornece listas de equipamentos e unidades, e recebe os dados de novos registros de movimento.
- configuracoes.jsp (Configurações) de perfis e permissões.
 - Função: ajustar preferências do usuário (tema, idioma, itens por página, dados de perfil).
 - Conexão com controlador: interagem principalmente com ConfiguracaoController para carregar parâmetros e dados do usuário.

4.1.3 Controlador

As classes Controller são responsáveis pelo fluxo de execução da aplicação, situadas no pacote `br.com.patrimweb.controller`. Elas recebem requisições HTTP mapeadas por

anotações @WebServlet ou por definições em web.xml, interpretam parâmetros, interagem com a camada Model/DAO e retornam respostas apropriadas para as views.

Principais controllers:

- LoginController.java
 - Recebe requisições POST com credenciais de login, utiliza UsuarioDAO para autenticação e, em caso de sucesso, cria uma HttpSession com o usuário autenticado.
 - Em caso de falha, retorna para index.jsp com mensagem de erro.
- LoginGoogleController.java
 - Recebe requisições POST com credenciais de login, utiliza OAuth do Google para autenticação e, em caso de sucesso, cria uma HttpSession com o usuário autenticado e insere o usuário na tabela usuário.
 - Em caso de falha, retorna para index.jsp e não conecta.
- LogoutController.java
 - Recebe requisições solicitando finalização de sessão para desconexão de usuário.
 - Em caso de falha, retorna para index.jsp com mensagem de erro.
- DashboardController.java
 - Trata requisições à página principal após login, agregando estatísticas (quantidade de equipamentos, movimentações, usuários) para o dashboard.
 - Encaminha os dados para dashboard.jsp via atributos de requisição.
- EquipamentoController.java
 - Centraliza operações CRUD de equipamentos, diferenciando ações por parâmetro action (listar, adicionar, editar, excluir).
 - Interage com EquipamentoDAO, FabricanteDAO e UnidadeDAO para compor a lista de equipamentos e opções de seleção nas views.
- FabricanteController.java
 - Controla o fluxo das operações de fabricantes, garantindo que exclusões respeitem as restrições de integridade referencial.
- UsuarioController.java
 - Gerencia a criação, edição, exclusão e listagem de usuários, além de interagir com RelatorioController para funcionalidades de relatório.
- UnidadeController.java
 - Responsável pelas operações CRUD sobre unidades organizacionais.
- MovimentacaoController.java
 - Gerencia o registro de movimentações, incluindo validação de dados de origem, destino e equipamento selecionado.
 - Consulta MovimentacaoDAO para listar o histórico de movimentações apresentado na interface.

- RelatorioFabricanteController.java
 - Concentra a geração de relatórios de fabricantes, recebendo filtros via parâmetros de requisição (período, fabricante) e retornando coleções de dados para views de relatório.
- RelatorioEquipamentoController.java
 - Concentra a geração de relatórios de equipamentos, recebendo filtros via parâmetros de requisição (período, nome_equip, fabricante) e retornando coleções de dados para views de relatório.
- RelatorioUnidadeController.java
 - Concentra a geração de relatórios de unidades, recebendo filtros via parâmetros de requisição (período, nome, endereço) e retornando coleções de dados para views de relatório.
- RelatorioUsuarioController.java
 - Concentra a geração de relatórios de usuários, recebendo filtros via parâmetros de requisição (período, nome, cpf) e retornando coleções de dados para views de relatório.
- RelatorioMovimentacaoController.java
 - Concentra a geração de relatórios de movimentações, recebendo filtros via parâmetros de requisição (período, tipo movimentação, equipamento/serial) e retornando coleções de dados para views de relatório.

Essas classes tipicamente implementam métodos doGet e doPost, nos quais validam sessões, instanciam DAOs, tratam exceções, definem atributos em request ou session e encaminham para JSPs via forward ou redirect.

4.2 TELAS

As telas serão listadas em ordem lógica de uso e descrição das ações disponíveis ao usuário.

1. Tela de Login – index.jsp

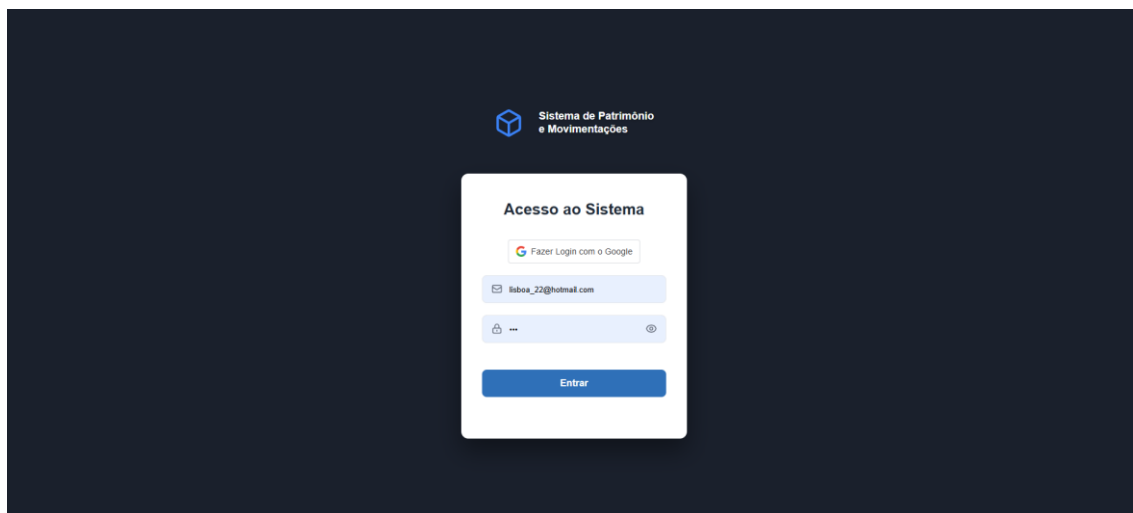


Figura 3 – Tela de Login.

Esta tela tem como objetivo autenticar o usuário no sistema, funcionando como ponto de entrada da aplicação. Apresenta campos para “E-mail” e “Senha”, botões de acesso, elementos de integração com Google Sign-In. O usuário pode inserir suas credenciais e acionar o botão “Entrar” ou “Fazer Login com Google”, o que dispara uma requisição POST ao LoginController ou LoginGoogleControle; em caso de falha, mensagens de erro são exibidas na própria tela.

2. Tela de Dashboard – dashboard.jsp

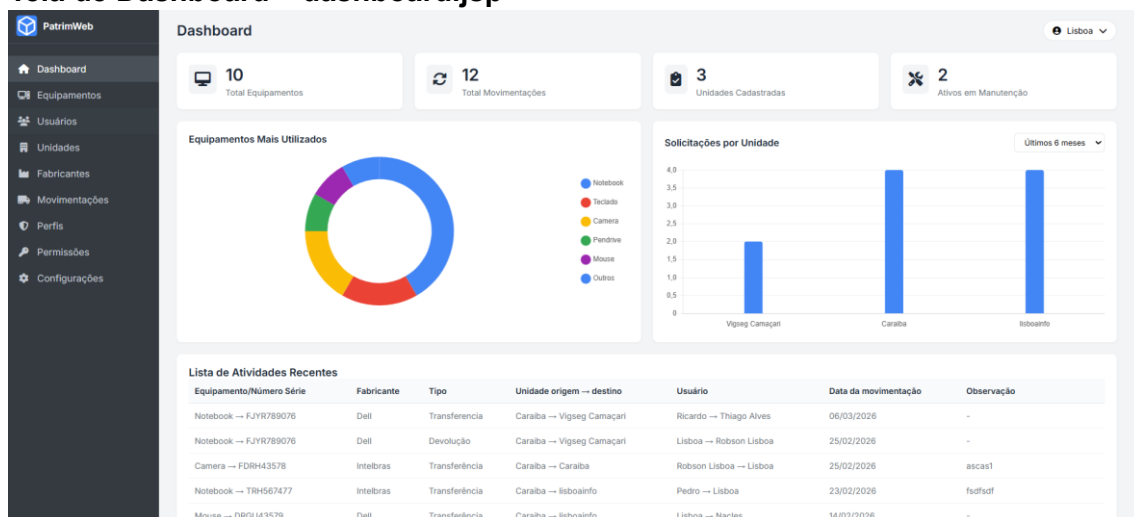


Figura 4 – Tela de Dashboard.

A tela de dashboard fornece uma visão geral do sistema após a autenticação, reunindo indicadores resumidos do patrimônio e facilitando a navegação entre módulos. Ela apresenta cabeçalho com informações do usuário logado, menu lateral com links para módulos (Equipamentos, Fabricantes, Usuários, Unidades, Movimentações, Perfis, Permissões e Configurações) e cards com estatísticas, além de uma tabela de últimas movimentações. As ações disponíveis incluem navegar para qualquer módulo principal, visualizar estatísticas principais e acessar opções de perfil ou logout.

3. Tela de Gerenciamento de Equipamentos – equipamentos.jsp

#ID	Nome do Equipamento	Numero de Série	Fabricante	Data Inserção	Ações
1	Notebook	FJYR789076	Dell	03/02/2026	
3	Pendrive	DGTR6885	Kingston	03/02/2026	
4	Mouse	DROU43579	Dell	03/02/2026	
6	Teclado	FHKU45788	Dell	05/02/2026	
7	Camera	FDRH43578	Intelbras	05/02/2026	
8	DVR	RFTG65788F	Hikvision	05/02/2026	
20	Notebook	TRH567477	Intelbras	23/02/2026	
21	Webcam	NHD5678JH	Intelbras	23/02/2026	
22	CD	NFTHJD	Intelbras	23/02/2026	
23	DVD	HFT6854HJL	TP_Link	23/02/2026	
26	Celular		Intelbras	11/03/2026	
28	Celular		Dell	11/03/2026	
29	Monitor		Dell	11/03/2026	
30	Monitor		Dell	11/03/2026	
31	Monitor		Dell	11/03/2026	

Figura 5 – Tela de Gerenciamento de Equipamento.

Esta tela é destinada ao cadastro e manutenção dos equipamentos cadastrados no patrimônio. O usuário pode visualizar uma tabela com colunas como Nome do Equipamento, Fabricante, Data de Cadastro, bem como utilizar filtros por nome ou fabricante. As ações incluem criar novo equipamento (via botão “Novo Equipamento” e formulário modal), editar registros existentes, excluir equipamentos e acessar o histórico de cadastro de um equipamento.

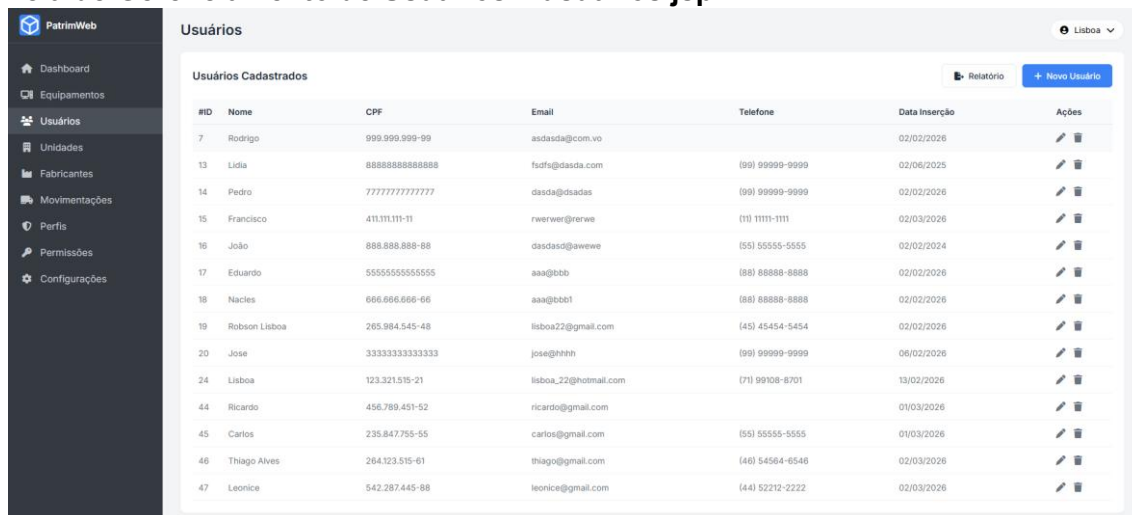
4. Tela de Gerenciamento de Fabricantes – fabricantes.jsp

#ID	Nome do Fabricante	Data de Inserção	Ações
2	Intelbras	02/02/2026	
3	Hikvision	02/02/2026	
4	TP_Link	02/02/2026	
5	Dell	02/02/2026	
8	Kingston	06/02/2026	
10	Samsung	11/02/2026	

Figura 5 – Tela de Gerenciamento de Fabricante.

A tela de fabricantes permite gerenciar a lista de fabricantes cadastrados, garantindo referência adequada para os equipamentos. Nela, o usuário encontra uma tabela com nome do fabricante, data de inserção e ações (editar, excluir), bem como o botão “Novo Fabricante” para inclusão de novos registros. As ações possíveis incluem cadastrar, alterar e excluir fabricantes, sendo que exclusões podem ser bloqueadas quando existirem equipamentos associados devido a restrições de chaves estrangeiras.

5. Tela de Gerenciamento de Usuários – usuarios.jsp

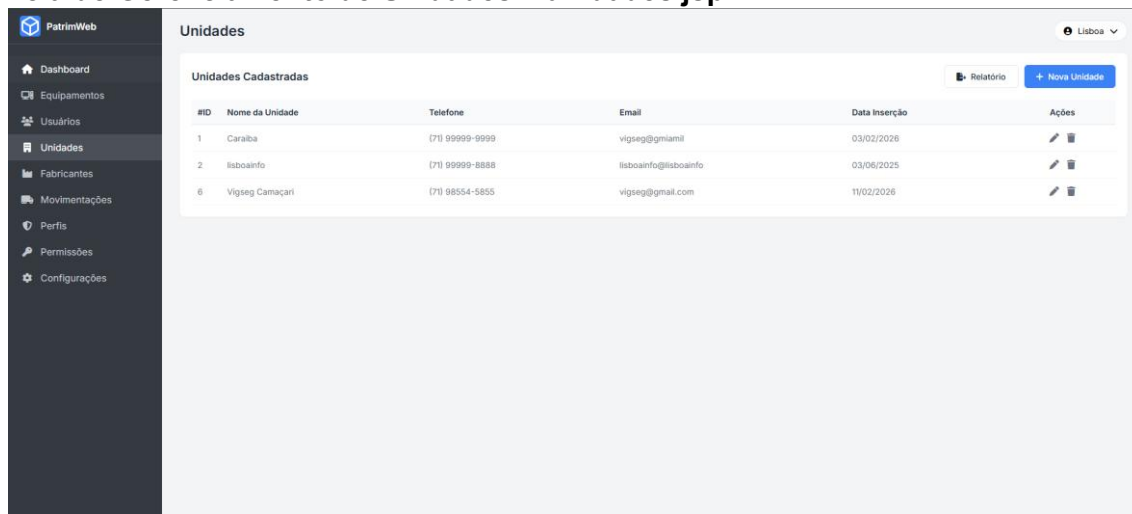


ID	Nome	CPF	Email	Telefone	Data Inserção	Ações
7	Rodrigo	999.999.999-99	asdasda@com.vo		02/02/2026	 
13	Lidia	88888888888888	fsdfs@dsada.com	(99) 99999-9999	02/06/2025	 
14	Pedro	77777777777777	dasda@dsadas	(99) 99999-9999	02/02/2026	 
15	Francisco	411.111.111-11	rwerwer@erwe	(11) 1111-1111	02/03/2026	 
16	João	888.888.888-88	dsdsad@awewe	(55) 55555-5555	02/02/2024	 
17	Eduardo	55555555555555	aaa@bbb	(88) 88888-8888	02/02/2026	 
18	Nacies	666.666.666-66	aaa@bbb1	(88) 88888-8888	02/02/2026	 
19	Robson Lisboa	265.984.545-48	lisboa22@gmail.com	(45) 45454-5454	02/02/2026	 
20	Jose	33333333333333	jose@hhhh	(99) 99999-9999	06/02/2026	 
24	Lisboa	123.321.515-21	lisboa_22@hotmail.com	(71) 99108-8701	13/02/2026	 
44	Ricardo	456.789.451-52	ricardo@gmail.com		01/03/2026	 
45	Carlos	235.847.755-55	carlos@gmail.com	(55) 55555-5555	01/03/2026	 
46	Thiago Alves	264.123.515-61	thiago@gmail.com	(46) 54564-6546	02/03/2026	 
47	Leonice	542.287.445-88	leonice@gmail.com	(44) 52212-2222	02/03/2026	 

Figura 6 – Tela de Gerenciamento de Usuário.

Esta tela é responsável por administrar os usuários do sistema. Ela apresenta link para relatório_usuarios que contém filtros (por período, nome, CPF) e uma tabela com colunas para Id, Nome, CPF, Telefone, Email, Endereço e Data de Inserção. O usuário pode realizar operações de inclusão (via “+ Novo Usuário”), edição de perfis, exclusão de registros e geração de relatórios exportáveis sobre usuários cadastrados.

6. Tela de Gerenciamento de Unidades – unidades.jsp



ID	Nome da Unidade	Telefone	Email	Data Inserção	Ações
1	Caraliba	(71) 99999-9999	vigseg@gniamil	03/02/2026	 
2	lisboainfo	(71) 99999-8888	lisboainfo@lisboainfo	03/06/2025	 
6	Vigseg Camaçari	(71) 98554-5855	vigseg@gmail.com	11/02/2026	 

Figura 7 – Tela de Gerenciamento de Unidade.

A tela de unidades controla os setores, departamentos ou locais que podem receber equipamentos. Uma tabela apresenta nome da unidade, telefone, email, data de inserção e ações de manutenção. As ações disponíveis incluem cadastro de novas unidades, edição de unidades existentes e exclusão, respeitando vínculos com equipamentos e movimentações, quando aplicável.

7. Tela de Movimentações – movimentacoes.jsp

ID	Equipamento/Número Série	Fabricante	Tipo	Origem → Destino	Libração → Recepção	Observação	Data	Ações
21	DVR → RFT065788F	Hikvision	Manutenção	Caralba → lisboainfo	Thiago Alves → Ricardo		09/03/2026	
20	Notebook → FUYR789076	Dell	Transferência	Caralba → Vigseg Camaçari	Ricardo → Thiago Alves		06/03/2026	
18	Camera → FDRH43578	Intelbras	Transferência	Caralba → Caralba	Robson Lisboa → Lisboa1	ascas1	25/02/2026	
17	Notebook → TRH567477	Intelbras	Transferência	Caralba → lisboainfo	Pedro → Lisboa1	fsdfsd	23/02/2026	
16	Mouse → DRGU43579	Dell	Transferência	Caralba → lisboainfo	Lisboa1 → Nacles		14/02/2026	
6	Teclado → FHKU45788	Dell	Transferência	Caralba → lisboainfo	Pedro → Nacles		05/09/2023	
1	Pendrivel → DQTR6885	Kingston	Manutenção	Caralba → lisboainfo	Robson Lisboa → Eduardo	dassas	04/02/2026	

Figura 8 – Tela de Gerenciamento de Movimentação.

Esta tela foi concebida para registrar e consultar movimentações de equipamentos entre unidades. O usuário pode acionar o botão “Nova Movimentação” para abrir formulário que recebe equipamento, fabricante, tipo de movimentação, unidade de origem, usuários de origem, unidade de destino, usuário de destino, observação e data/hora de inserção (geralmente preenchida automaticamente). Também há uma tabela de histórico, na qual é possível consultar o percurso dos equipamentos, filtrar por equipamento ou período e executar ações de edição ou exclusão de movimentações.

8. Tela de Perfil – perfis.jsp

ID	Nome do Perfil	Ações
1	ADMINISTRADOR	
2	TECNICO	
3	OPERADOR	

Figura 9 – Tela de Gerenciamento de Perfil.

Esta tela foi concebida para registrar e consultar perfis de equipamentos. O usuário pode acionar o botão “+ Novo Perfil” para abrir formulário que recebe perfil e data de inserção (geralmente preenchida automaticamente).

9. Tela de Permissão – permissao.jsp

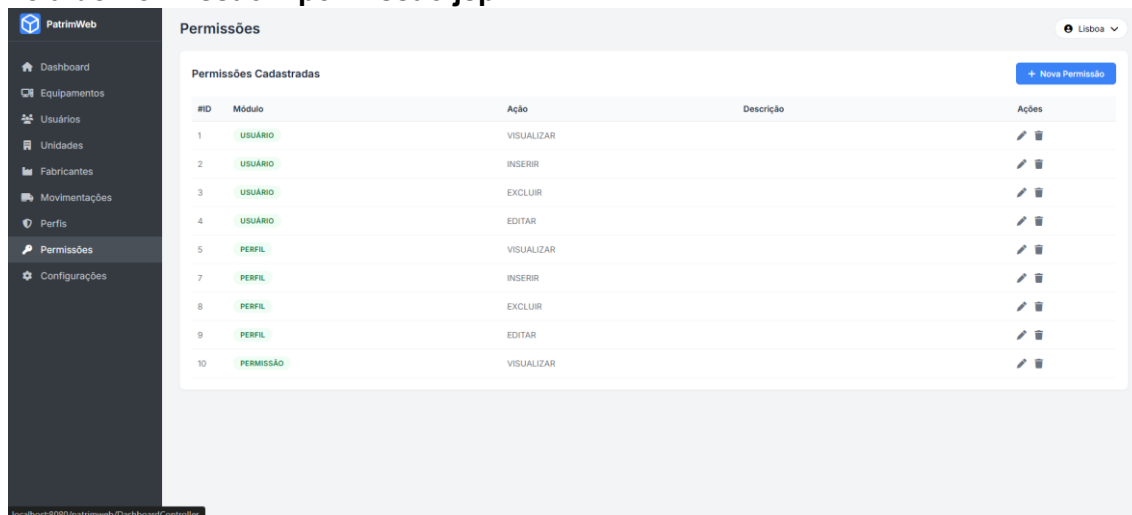


Figura 10 – Tela de Gerenciamento de Permissão.

Esta tela foi concebida para registrar permissões de usuários. O usuário pode acionar o botão “Nova Permissão” para abrir formulário que recebe permissão e data de inserção (geralmente preenchida automaticamente).

10. Tela de Configurações – configuracoes.jsp

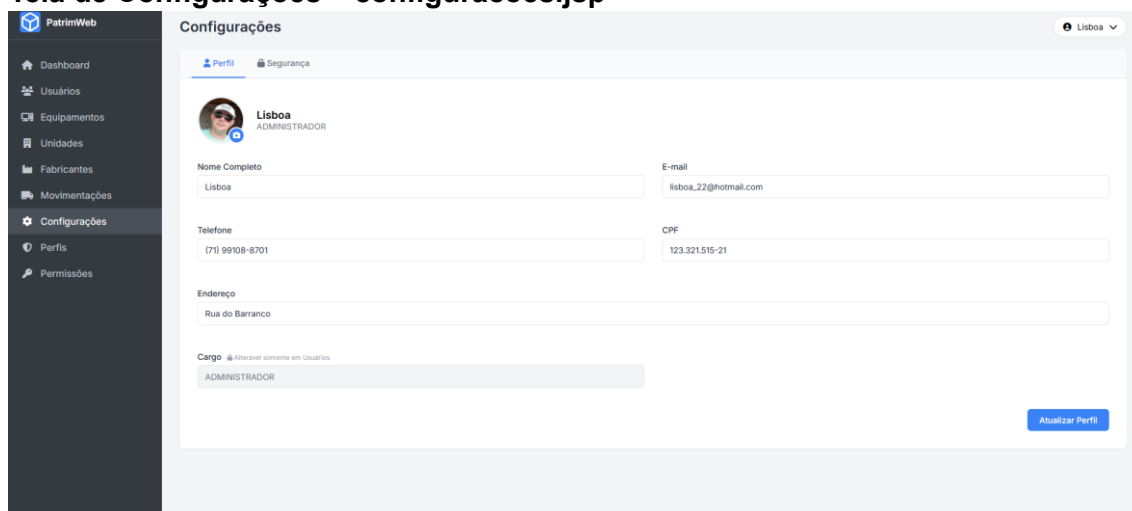


Figura 11 – Tela de Configurações.

A tela de configurações concentra informações sobre o usuário logado e permite alterações no perfil e na foto do perfil.

11. Telas de Relatórios de Fabricante – (relatório de fabricantes)

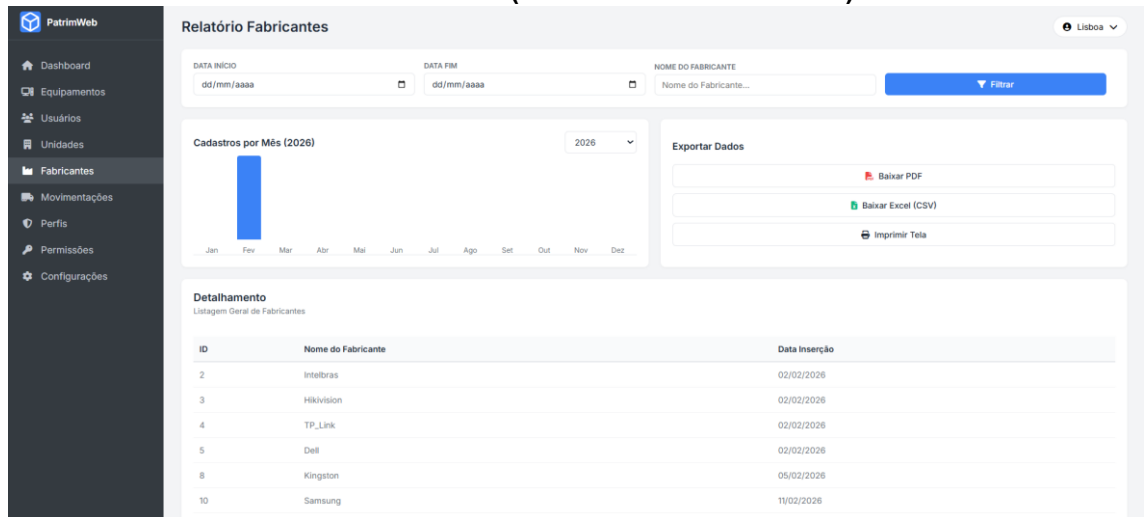


Figura 12 – Tela de Relatório de Fabricantes.

A tela de relatório fabricante apresenta dados consolidados para análise gerencial. Nela, o usuário informa filtros (por período, nome) e, ao acionar o botão filtrar, o sistema gera tabelas de acordo com o filtro escolhido. Traz também gráficos ou opções de exportação (por exemplo, para PDF ou planilha CSV). As principais ações são configurar filtros, imprimir tela, exportar resultados em arquivos nos formatos citados acima, e gráficos de análises por ano e mês.

12. Telas de Relatórios de Equipamentos – (relatório de equipamentos)

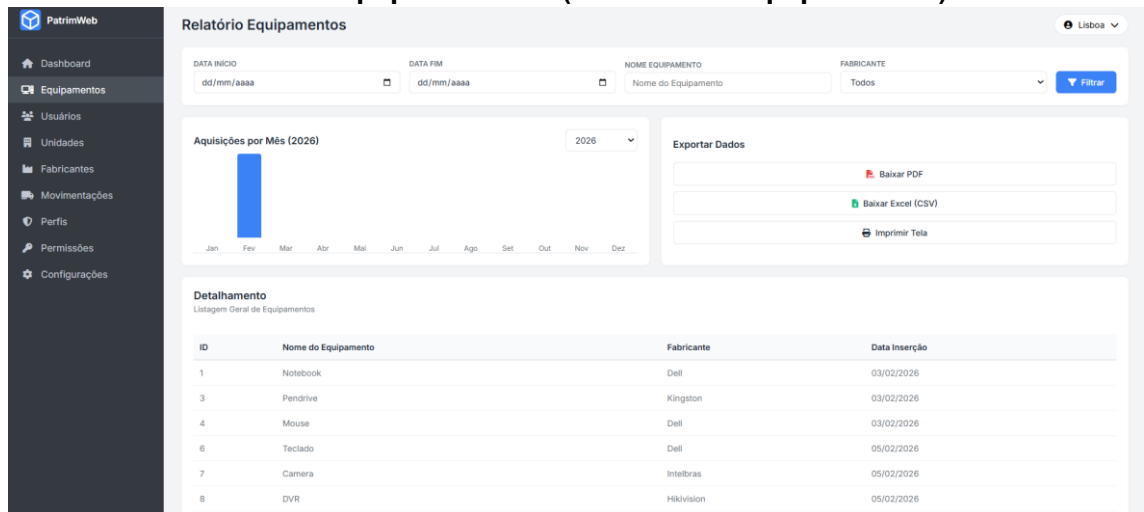


Figura 13 – Tela de Relatório de Equipamentos.

A tela de relatório equipamentos apresenta dados consolidados para análise gerencial. Nela, o usuário informa filtros (por período, nome, fabricante) e, ao acionar o botão filtrar, o sistema gera tabelas filtradas de acordo com o filtro escolhido. Traz também gráficos ou opções de exportação (por exemplo, para PDF ou planilha CSV). As principais ações são configurar filtros, imprimir tela, exportar resultados em arquivos nos formatos citados acima, e gráficos de análises por ano e mês.

13. Telas de Relatórios de Usuários – (relatório de usuários)

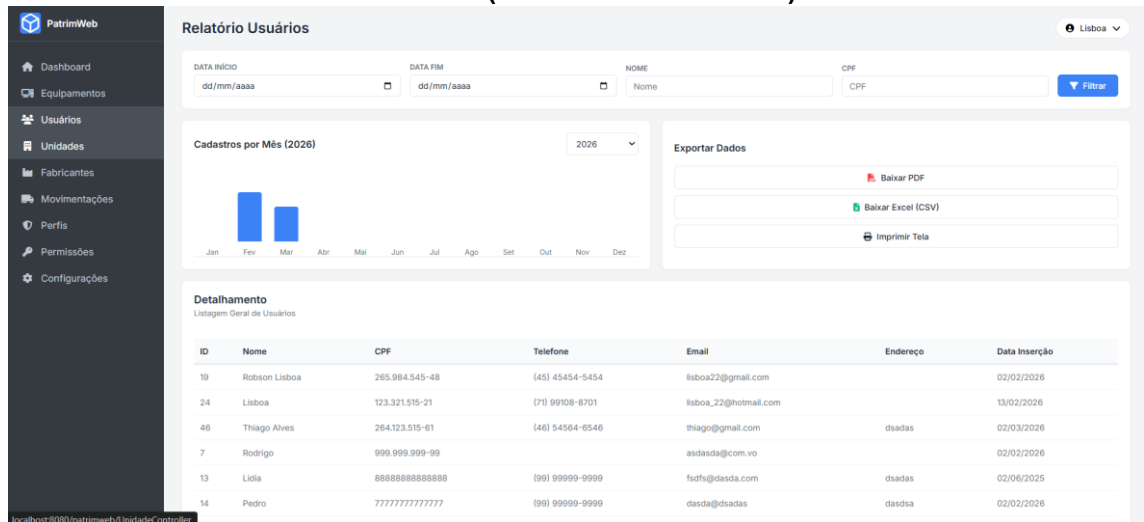


Figura 14 – Tela de Relatórios Usuários.

A tela de relatório fabricante apresenta dados consolidados para análise gerencial. Nela, o usuário informa filtros (por período, nome, cpf) e, ao acionar o botão filtrar, o sistema gera tabelas filtradas de acordo com o filtro escolhido. Traz também gráficos ou opções de exportação (por exemplo, para PDF ou planilha CSV). As principais ações são configurar filtros, imprimir tela, exportar resultados em arquivos nos formatos citados acima, e gráficos de análises por ano e mês.

14. Telas de Relatórios de Unidades – (relatório de unidades)

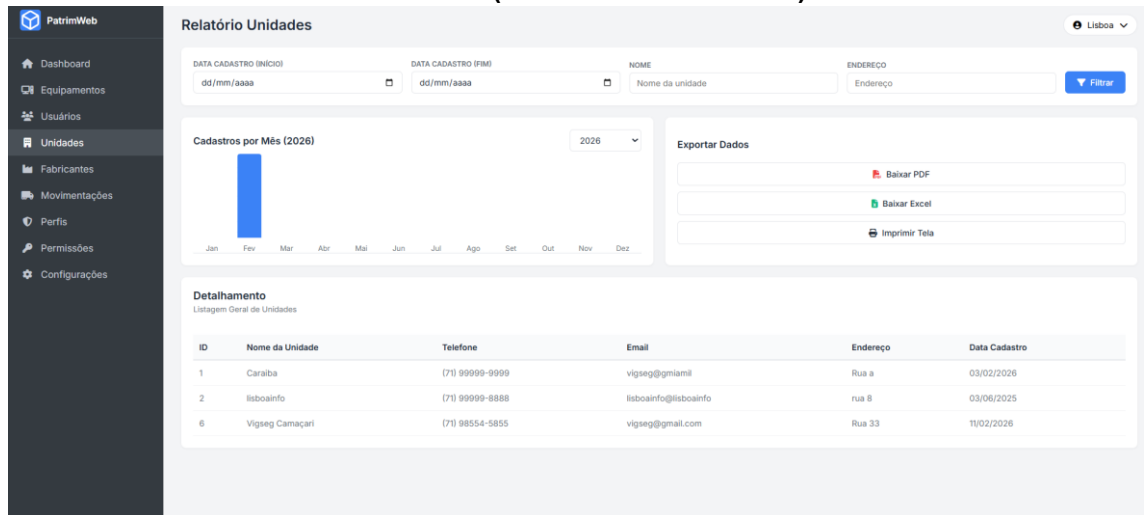


Figura 15 – Tela de Relatório de Unidades.

A tela de relatório fabricante apresenta dados consolidados para análise gerencial. Nela, o usuário informa filtros (por período, nome, endereço) e, ao acionar o botão filtrar, o sistema gera tabelas filtradas de acordo com o filtro escolhido. Traz também gráficos ou opções de exportação (por exemplo, para PDF ou planilha CSV). As principais ações são configurar filtros, imprimir tela, exportar resultados em arquivos nos formatos citados acima, e gráficos de análises por ano e mês.

15. Telas de Relatórios de Movimentações – (relatório de movimentações)

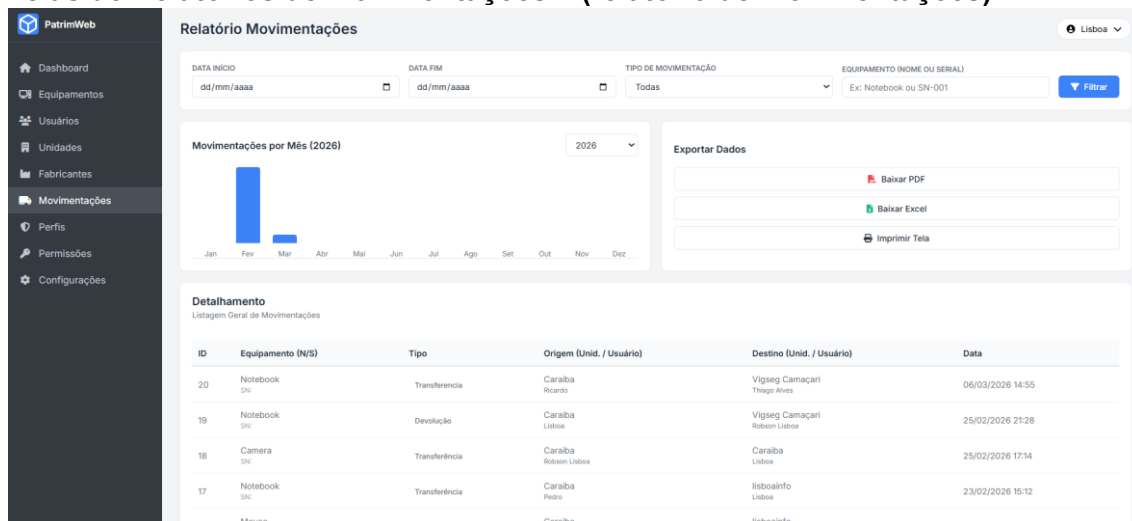


Figura 16 – Tela de Relatório de Movimentações.

A tela de relatório movimentações apresenta dados consolidados para análise gerencial. Nela, o usuário informa filtros (por período, tipo movimentação, Equipamento/Número Série) e, ao acionar o botão filtrar, o sistema gera tabelas filtradas de acordo com o filtro escolhido. Traz também gráficos ou opções de exportação (por exemplo, para PDF ou planilha CSV). As principais ações são configurar filtros, imprimir tela, exportar resultados em arquivos nos formatos citados acima, e gráficos de análises por ano e mês.

CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema PatrimWeb possibilitou a aplicação prática de diversos conceitos estudados ao longo da disciplina de Laboratório de Programação I, envolvendo arquitetura de sistemas, modelagem de banco de dados, desenvolvimento web e organização de código em camadas. A utilização do padrão arquitetural MVC contribuiu para a separação clara de responsabilidades entre as camadas de apresentação, controle e persistência, favorecendo a manutenção, escalabilidade e organização do projeto.

Durante a implementação, foram utilizadas tecnologias amplamente empregadas no desenvolvimento de aplicações corporativas, como Java, JSP, Servlets e o banco de dados MySQL. A integração entre essas tecnologias permitiu construir uma aplicação web funcional capaz de gerenciar informações patrimoniais de forma estruturada, segura e com histórico rastreável de movimentações de equipamentos entre unidades organizacionais.

O sistema desenvolvido contempla funcionalidades essenciais para o gerenciamento patrimonial, incluindo cadastro e manutenção de equipamentos, fabricantes, usuários e unidades, além do registro detalhado de movimentações e geração de relatórios gerenciais. Esses recursos permitem maior controle sobre os ativos da organização, facilitando processos de auditoria, acompanhamento de localização de equipamentos e análise de dados operacionais.

Outro aspecto relevante do projeto foi a implementação de perfis e permissões de acesso, o que contribui para maior segurança e controle das ações executadas no sistema. A interface web desenvolvida por meio de JSP, HTML, CSS e JavaScript proporciona uma navegação organizada entre os módulos, permitindo que usuários realizem operações de forma intuitiva.

Como possibilidades de evolução futura, o sistema pode ser ampliado com funcionalidades adicionais, como integração com leitores de código de barras ou QR Code para identificação de equipamentos, geração de relatórios mais avançados com ferramentas de Business Intelligence, integração com APIs externas e melhorias na interface do usuário utilizando frameworks modernos de front-end.

Dessa forma, o projeto PatrimWeb demonstra a viabilidade de uma solução web para gerenciamento de patrimônio institucional, ao mesmo tempo em que consolida conhecimentos técnicos importantes na formação acadêmica na área de computação e desenvolvimento de sistemas.

Link Github: <https://github.com/lisboa22/PatrimWeb>